

Prompting Fundamentals on Real Tasks v2

Ahmed Elgharably — FlyRank ML Internship — General AI Fluency, Week 2 (FL-02)

The Real Task

Task: Build a content refresh scoring model for the FlyRank starter dataset (`content_refresh_anonymized.csv`).

Why this task: This is exactly what I'm doing for my ML track capstone (Lane 2: Refresh / Content Opportunity Scoring). The dataset has 30,000 rows of content performance data. I need a model that ranks pages by refresh priority so a content team with limited capacity knows which 20–50 pages to review first each week.

Tools used for this assignment:

- Claude 3.5 Sonnet (via Claude.ai)
 - ChatGPT-4o (via ChatGPT)
-

Version 0: The Naive Prompt

Prompt:

Build a content refresh scoring model.

Output (Claude):

I'd be happy to help you build a content refresh scoring model! To get started, I'll need some information:

1. What data do you have available? (traffic metrics, engagement data, content age, etc.)
2. What format is your data in?
3. What's your definition of "refresh-worthy"? (declining traffic, old content, low engagement?)
4. What tools are you using? (Python, R, Excel?)
5. How will the output be used? (automated flagging, weekly report, dashboard?)

Once you provide these details, I can guide you through building a scoring model step by step.

What went wrong: The AI had no idea what I was working with. It asked five clarifying questions instead of producing anything useful. I got a conversation, not a model. This is what I would have typed a month ago.

Version 1: Role Assignment

Technique applied: Role assignment — telling the AI who it is.

Prompt:

You are an ML engineer at a digital marketing analytics company. Build a content refresh scoring model.

Output (Claude):

As an ML engineer, here's how I'd approach building a content refresh scoring model:

1. Problem Framing We need to rank content pages by their likelihood of benefiting from an update. The target audience is a content operations team with limited review capacity.

2. Data Requirements Typical signals I'd expect:

- Traffic decay metrics (impressions, clicks, sessions over time)
- Content metadata (publish date, word count, content type)
- Engagement signals (bounce rate, time on page, scroll depth)
- Search performance (average position, CTR)

3. Model Architecture I'd recommend a gradient-boosted tree model (XGBoost or LightGBM) because:

- It handles mixed data types well
- Feature importance is interpretable
- It performs well on tabular data

4. Feature Engineering

- Trend features: 30-day vs 90-day traffic ratios
- Decay features: days since last update, content age
- Engagement ratios: CTR, engagement rate
- Volume thresholds: minimum impressions to avoid noise

5. Validation Strategy

- Time-based split (train on older data, test on newer)
- Client-level holdout if multi-tenant
- Precision@K for ranked output evaluation

Would you like me to write the actual code? If so, share your dataset structure.

What changed in the output: The AI stopped asking basic questions and started acting like an ML engineer. It produced a structured plan with specific technical recommendations (XGBoost, time-based splits, Precision@K) that an ML engineer would actually use. It assumed I needed a ranked output for a team with limited capacity — a detail it inferred from the “ML engineer at a digital marketing company” role.

What still failed: Still no code. Still no idea what my actual data looks like. The plan is generic — it mentions “typical signals” and “I'd expect” instead of working with my real columns. It's a good outline, but I can't run it.

What I would try next: Give it the actual dataset context so it can write runnable code instead of a theoretical plan.

Version 2: Context and Motivation

Technique applied: Context and motivation — giving it the real data structure and why I need this.

Prompt:

You are an ML engineer at a digital marketing analytics company. Build a content refresh scoring model.

Dataset: `content_refresh_anonymized.csv` with 30,000 rows. Key columns: `content_id`, `client_id`, `impressions_90d`, `clicks_90d`, `sessions_90d`, `avg_position`, `ctr`, `content_age_days`, `days_since_last_update`, `word_count`, `content_type`, `trend_direction` (up/down/stable), `engage-`

ment_rate, scroll_rate.

Why I need this: A content team can only review 20–50 pages per week. They need a ranked queue to know which pages to look at first. The team can take these actions: refresh content, expand thin sections, protect a declining winner, prune obsolete pages, or monitor and wait.

Output (Claude):

```
Got it — you have a real dataset and a capacity constraint. Here's a complete Python pipeline:
python > import pandas as pd > import numpy as np > from sklearn.ensemble import RandomForestClassifier > from sklearn.model_selection import train_test_split >
from sklearn.metrics import roc_auc_score, precision_score > > # Load data > df = pd.read_csv('content_refresh_anonymized.csv') > > # Define target: declining pages >
df['needs_refresh'] = (df['trend_direction'] == 'down').astype(int) > > # Features > features = ['impressions_90d', 'clicks_90d', 'sessions_90d', 'avg_position', >
'ctr', 'content_age_days', 'days_since_last_update', 'word_count', > 'engagement_rate', 'scroll_rate'] > > # Encode categorical > df = pd.get_dummies(df,
columns=['content_type'], drop_first=True) > features += [c for c in df.columns if c.startswith('content_type_')] > > # Split > X = df[features] > y = df['needs_refresh']
> X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
> > # Model > model = RandomForestClassifier(n_estimators=100, random_state=42) > model.fit(X_train, y_train) > > # Predict and rank > df_test = df.iloc[X_test.index].copy() >
df_test['refresh_prob'] = model.predict_proba(X_test)[ :, 1] > top_50 = df_test.nlargest(50, 'refresh_prob') > > print(f"ROC AUC: {roc_auc_score(y_test, df_test['refresh_prob']):.3f}")
> print(f"Precision@50: {y_test.iloc[top_50.index].mean():.3f}") >
```

Why this works for your team:

- The model learns which signal combinations predict decline
- Precision@50 tells you: of the top 50 recommendations, how many are actually declining?
- The team gets a ranked list they can work through week by week

Caveat: This uses `trend_direction` as the target, which is a current-state label, not a future outcome. For a stronger capstone, you'd want to predict “will this page decline in the next 30 days?” using a time-windowed label.

What changed in the output: Now I got actual runnable Python code using my real column names. The AI wrote a full pipeline: load data, define target, select features, encode categoricals, train a Random Forest, predict probabilities, and rank. It also added a relevant metric (Precision@50) and a honest caveat about the target being a proxy label — something an ML engineer would flag.

What still failed: The train/test split is random, not client-holdout. The assignment guide warns that pages from the same client share patterns, so a random split lets the model “cheat” by memorizing client-specific behavior. Also, there's no leakage audit, no baseline comparison, and no reason codes for the reviewer. The code works, but it's not rigorous enough for a capstone.

What I would try next: Ask for a structured output format that includes validation rigor and human-readable explanations.

Version 3: Output Structure

Technique applied: Output structure — specifying exactly what the response should look like.

Prompt:

You are an ML engineer at a digital marketing analytics company. Build a content refresh scoring model.

Dataset: [same as V2] **Why I need this:** [same as V2]

Output format: Provide your response in exactly these sections:

1. **Data Contract** — which columns are features, which is the target, and which are excluded
2. **Leakage Audit** — list any features that might leak future information or the target
3. **Validation Design** — specify the split strategy (must be client-level holdout, not random)
4. **Baseline** — write a transparent rule-based score to compare against
5. **Model Code** — the full Python pipeline
6. **Evaluation** — report ROC AUC, Average Precision, and Precision@50
7. **Reason Codes** — add human-readable tags explaining why each top page was flagged

Output (Claude):

1. Data Contract

Features: impressions_90d, clicks_90d, sessions_90d, avg_position, ctr, content_age_days, days_since_last_update, word_count, engagement_rate, scroll_rate, content_type (encoded) **Target:** is_declining = (trend_direction == 'down').astype(int) **Excluded:** content_id (identifier), client_id (used for grouping only)

2. Leakage Audit

- trend_direction is the target — excluded from features
- trend_pct is not in the dataset
- No future-dated columns present
- All features are observable at prediction time

3. Validation Design

Client-level GroupShuffleSplit: entire clients held out from training. The model is tested on clients it has never seen, preventing memorization of client-specific patterns.

4. Baseline

```
python > baseline_score = ( > 0.40 * np.clip(df['impressions_90d'] / 1000, 0, 1) + >
0.30 * (df['content_age_days'] >= 180).astype(float) + > 0.25 * (df['avg_position']
<= 10).astype(float) * (df['ctr'] < 0.5).astype(float) + > 0.05 * (df['word_count'] <
1200).astype(float) > ) >
```

5. Model Code

```
python > from sklearn.model_selection import GroupShuffleSplit > from sklearn.ensemble
import RandomForestClassifier > from sklearn.metrics import roc_auc_score, aver-
age_precision_score > gss = GroupShuffleSplit(n_splits=1, test_size=0.3, random_state=42)
> train_idx, test_idx = next(gss.split(df, groups=df['client_id'])) > X_train,
X_test = df.iloc[train_idx][features], df.iloc[test_idx][features] > y_train, y_test =
df.iloc[train_idx]['is_declining'], df.iloc[test_idx]['is_declining'] > model = Ran-
```

```
domForestClassifier(n_estimators=100, random_state=42) > model.fit(X_train, y_train) > >
test_df = df.iloc[test_idx].copy() > test_df['model_prob'] = model.predict_proba(X_test)[: ,
1] > test_df['baseline_prob'] = baseline_score.iloc[test_idx] >
```

6. Evaluation

```
python > def precision_at_k(y_true, probs, k=50): >         top_k = np.argsort(probs)[-k:] >
return y_true.iloc[top_k].mean() > > print(f"Baseline — ROC AUC: {roc_auc_score(y_test,
test_df['baseline_prob']):.3f}") > print(f"Baseline — Precision@50: {precision_at_k(y_test,
test_df['baseline_prob']):.3f}") > print(f"Model — ROC AUC: {roc_auc_score(y_test,
test_df['model_prob']):.3f}") > print(f"Model — Precision@50: {precision_at_k(y_test,
test_df['model_prob']):.3f}") >
```

7. Reason Codes

```
python > def get_reason_codes(row): >         codes = [] >         if row['trend_direction'] ==
'down' and row['impressions_90d'] >= 100: >             codes.append('declining_with_demand')
>         if row['content_age_days'] >= 180 and row['impressions_90d'] >= 500: >
codes.append('stale_visible_page') >         if row['avg_position'] <= 20 and row['ctr'] <
0.5 and row['impressions_90d'] >= 500: >             codes.append('low_ctr_opportunity') >
return ', '.join(codes) if codes else 'model_flagged' > > top_50 = test_df.nlargest(50,
'model_prob') > top_50['reason_codes'] = top_50.apply(get_reason_codes, axis=1) >
```

What changed in the output: The response is now perfectly structured. Every section I asked for is present and filled with real, runnable code. The validation design uses client-level holdout (GroupShuffle-Split). The baseline is transparent and comparable. The reason codes give human reviewers explanations. This is capstone-quality output.

What still failed: The model code is correct but the baseline weights (0.40, 0.30, 0.25, 0.05) are arbitrary — the AI invented them without justification. Also, the reason codes use fixed thresholds (180 days, 500 impressions) that might not be optimal for all content types. The AI didn't show me HOW it arrived at these numbers.

What I would try next: Add few-shot examples showing what optimal threshold logic looks like, or decompose the problem into steps so the AI justifies each threshold.

Version 4: Few-Shot Examples

Technique applied: Few-shot examples — showing the AI what good output looks like.

Prompt:

[Same context and output structure as V3]

Here is an example of a well-justified threshold from the FlyRank starter guide: python > # Threshold justification: a page needs at least 500 impressions in 90 days > # to be "visible enough to matter." Below that, any decline is likely noise. > MIN_IMPRESSIONS_VISIBLE = 500 > > # A page is "stale" if it hasn't been updated in 180+ days. > # This matches the industry standard for content freshness audits. > STALE_THRESHOLD_DAYS = 180 >

Here is an example of a poorly justified threshold: python > # Bad: no justification, arbitrary

```
number > if impressions > 347: # why 347? No one knows. >
```

Apply this standard: every threshold must have a one-line justification tied to either the data distribution or the business context.

Output (Claude):

1. Data Contract

[Same structure, explicit about all choices]

2. Leakage Audit

[Same, plus note: `days_since_last_update` is safe because it's content metadata known at prediction time]

4. Baseline (with justifications)

```
python > # Thresholds justified by data + business context: > MIN_IMPRESSIONS_VISIBLE = 500
# Pages below this have too little traffic to justify review effort > STALE_DAYS = 180
# 6 months: standard content freshness window > PAGE_ONE_POSITION = 10 # Position 1-10 = page one of Google; high visibility, high opportunity cost > THIN_CONTENT_WORDS = 1200
# Below this, content is typically too thin to rank well for competitive terms >
> baseline_score = ( > 0.40 * np.clip(df['impressions_90d'] / MIN_IMPRESSIONS_VISIBLE, 0, 1) + > 0.30 * (df['content_age_days'] >= STALE_DAYS).astype(float) + > 0.25 * (df['avg_position'] <= PAGE_ONE_POSITION).astype(float) * > (df['ctr'] < 0.5).astype(float) + > 0.05 * (df['word_count'] < THIN_CONTENT_WORDS).astype(float) > )
> Why these weights: Visibility matters most (40%) because a page must have audience to be worth refreshing. Staleness is next (30%) because old content loses relevance. CTR opportunity on page one is high-impact but narrow (25%). Thin content is a weak signal alone (5%).
```

7. Reason Codes (with justifications)

```
python > def get_reason_codes(row): > codes = [] > # Declining + demand: trend is down AND enough volume to be real (not noise) > if row['trend_direction'] == 'down' and row['impressions_90d'] >= 100: > codes.append('declining_with_demand')
> # Stale + visible: old enough to be stale AND visible enough to matter > if row['content_age_days'] >= STALE_DAYS and row['impressions_90d'] >= MIN_IMPRESSIONS_VISIBLE:
> codes.append('stale_visible_page') > # Low CTR on page 1-2: ranks well but under-captures clicks > if (0 < row['avg_position'] <= 20 and row['ctr'] < 0.5 and > row['impressions_90d'] >= MIN_IMPRESSIONS_VISIBLE):
> codes.append('low_ctr_opportunity') > return ', '.join(codes) if codes else 'model_flagged' >
```

What changed in the output: Every threshold now has a justification. `MIN_IMPRESSIONS_VISIBLE = 500` is explained as “too little traffic to justify review effort.” `STALE_DAYS = 180` is tied to the “standard content freshness window.” The weights in the baseline have rationale. The few-shot examples trained the AI to explain its reasoning, not just throw numbers.

What still failed: The code is still delivered as one big block. A junior engineer might struggle to follow the logic flow. Also, there's no error handling — what if `content_type` has missing values? What if `impressions_90d` has zeros and we try to divide? The AI assumed clean data.

What I would try next: Break the task into steps so the AI handles edge cases and explains the logic flow.

Version 5: Step Decomposition

Technique applied: Step decomposition — breaking the task into ordered steps with checkpoints.

Prompt:

[Same context, structure, and few-shot standard as V4]

Step-by-step instructions: Build this in exactly 5 steps. Do not skip steps. After each step, pause and confirm the logic is sound before proceeding.

Step 1: Load and validate the data. Handle missing values, check for data quality issues, and report the shape and any anomalies. **Step 2:** Define the target and feature set. Perform the leakage audit. Justify every exclusion. **Step 3:** Build the baseline score with justified thresholds. Test it and report baseline metrics. **Step 4:** Train the ML model using client-level holdout. Compare model metrics to baseline. **Step 5:** Generate the final ranked queue with reason codes. Add error handling for edge cases (missing values, zero impressions, unknown content types).

Output (Claude):

Step 1: Load and Validate

```
python > import pandas as pd > import numpy as np > > df =
pd.read_csv('content_refresh_anonymized.csv') > print(f"Shape: {df.shape}") >
print(f"Missing values per column:") > print(df.isnull().sum()[df.isnull().sum() >
0]) > > # Data quality checks > assert df['content_id'].is_unique, "Duplicate content_ids
found" > assert (df['impressions_90d'] >= 0).all(), "Negative impressions detected" >
assert df['trend_direction'].isin(['up', 'down', 'stable']).all(), "Invalid trend values"
> print("Data validation passed.") > Result: 30,000 rows × 44 columns. No missing values in
key columns. 3 invalid avg_position values (negative) — we'll clip to 0.
```

Step 2: Target and Features + Leakage Audit

```
python > # Target > df['is_declining'] = (df['trend_direction'] == 'down').astype(int)
> print(f"Declining rate: {df['is_declining'].mean():.1%}") > > # Features > features =
['impressions_90d', 'clicks_90d', 'sessions_90d', 'avg_position', >
'ctr',
'content_age_days', 'days_since_last_update', 'word_count', >
'engagement_rate',
'scroll_rate'] > > # Leakage audit > assert 'trend_direction' not in features, "LEAKAGE:
target in features" > assert 'is_declining' not in features, "LEAKAGE: derived target
in features" > print("Leakage audit passed.") > Result: Declining rate is 28.4%. Class is
imbalanced but workable.
```

Step 3: Baseline

[Justified baseline code with constants defined upfront] **Result:** Baseline ROC AUC = 0.627, Precision@50 = 0.240

Step 4: Model Training

[Client-holdout split, Random Forest, full metrics] **Result:** Model ROC AUC = 0.750, Precision@50 = 0.740 **Interpretation:** The model beats baseline by 3× on Precision@50. 37 of the top 50 recommendations are actually declining pages.

Step 5: Final Queue with Error Handling

```
python > def safe_reason_codes(row): >     codes = [] >     # Handle missing values safely >     impressions = row.get('impressions_90d', 0) or 0 >     age = row.get('content_age_days', 0) or 0 >     position = row.get('avg_position', 999) or 999 >     ctr = row.get('ctr', 0) or 0 >     trend = row.get('trend_direction', 'unknown') > >     if trend == 'down' and impressions >= 100: >         codes.append('declining_with_demand') >         if age >= 180 and impressions >= 500: >             codes.append('stale_visible_page') >         if 0 < position <= 20 and ctr < 0.5 and impressions >= 500: >             codes.append('low_ctr_opportunity') >     return ', '.join(codes) if codes else 'model_flagged' > > top_50['reason_codes'] = top_50.apply(safe_reason_codes, axis=1) > print(top_50[['content_id', 'model_prob', 'reason_codes']].head(10)) > Result: Top 10 pages flagged with clear reason codes. No crashes on edge cases.
```

What changed in the output: The AI now produces a **narrative pipeline** with checkpoints and error handling. It catches data quality issues (negative positions), reports intermediate results (declining rate, baseline metrics), and uses `.get()` with defaults for safe reason code generation. The step-by-step format makes it easy to debug — if Step 3 fails, I know the problem is in the baseline, not the model.

What still failed: The model training step uses default Random Forest hyperparameters. It didn't suggest tuning `max_depth` or `min_samples_leaf` to prevent overfitting, especially with client-holdout validation where some clients might have very few pages. Also, it didn't mention feature importance analysis, which would help explain WHY the model beats the baseline.

What I would try next: Ask for feature importance and hyperparameter rationale, but that's beyond the scope of this assignment. The current output is strong enough for a capstone starting point.

Cross-Model Comparison: Claude vs. ChatGPT

I ran the **final prompt (Version 5)** on both Claude 3.5 Sonnet and ChatGPT-4o.

Claude 3.5 Sonnet Output

- **Tone:** Methodical, cautious, educational. It explains WHY each step matters.
- **Accuracy:** Correctly used `GroupShuffleSplit` for client holdout. Caught the negative `avg_position` values. Added a leakage audit.
- **Structure:** Followed the 5-step decomposition exactly. Each step had code + result interpretation.
- **Failure point:** Didn't question whether `trend_direction == 'down'` is the right target. It accepted my proxy label without pushing back.
- **Best for:** Rigor, completeness, and when you need the AI to act like a careful peer reviewer.

ChatGPT-4o Output

- **Tone:** Direct, action-oriented, confident. It gets to the code faster.

- **Accuracy:** Also used `GroupShuffleSplit` correctly. But it missed the negative `avg_position` issue entirely — it just used the column as-is. No data quality checks.
- **Structure:** Followed the steps but with less narrative. More code, fewer explanations between steps.
- **Failure point:** It suggested `StandardScaler` and `PCA` for feature preprocessing — completely unnecessary for a Random Forest and potentially harmful (trees don't need scaling, `PCA` destroys interpretability). This is a “sounds smart but wrong” moment.
- **Best for:** Speed and getting a first draft fast, but you **MUST** verify every technical suggestion.

Honest Comparison

Dimension	Claude 3.5	ChatGPT-4o	Winner
Data quality checks	Caught negative positions	Missed it	Claude
Leakage awareness	Explicit audit	Mentioned briefly	Claude
Code correctness	Correct, no unnecessary steps	Added useless <code>PCA/Scaling</code>	Claude
Explanation depth	Step-by-step with interpretation	Code-heavy, light on why	Claude
Speed to output	Slower, more thorough	Faster, more concise	ChatGPT
Target critique	Accepted proxy without question	Same	Tie (both failed)
Reason codes	Well-justified thresholds	Similar, less justification	Claude

Verdict: For ML engineering tasks where correctness matters more than speed, Claude is the safer choice. ChatGPT is fine for brainstorming or first drafts, but its “sounds smart but wrong” suggestions (`PCA` for trees) could waste hours if I didn't catch them. I would use Claude for the final pipeline and ChatGPT only for quick syntax checks.

Final Reusable Prompt Template

Role: You are an ML engineer building a [TASK: scoring model / classifier / recommender] for [DOMAIN: content refresh / churn prediction / fraud detection].

Dataset: [FILE] with [N] rows. Key columns: [LIST].

Business context: [WHO] needs [OUTPUT] because [CONSTRAINT / CAPACITY]. They can take these actions: [LIST].

Build this in [N] steps:

1. Load and validate: handle missing values, check data quality, report anomalies
2. Define target and features: perform a leakage audit, justify exclusions
3. Build a transparent baseline with justified thresholds
4. Train the model using [VALIDATION STRATEGY: client-holdout / time-based / stratified]
5. Generate ranked output with human-readable reason codes and error handling

Quality standard: Every threshold must have a one-line justification. Every metric must be compared against the baseline. Use careful language: this is decision-support, not proof.

What I Learned

1. **Role assignment alone** turns a Q&A session into a structured plan. Without it, the AI asks what you want. With it, it acts like a specialist.
2. **Context is the biggest multiplier.** The jump from V1 to V2 (adding dataset + motivation) was larger than any other single change. The AI went from “here’s a general approach” to “here’s your code.”
3. **Output structure forces completeness.** V3’s section-by-section format made the AI include validation and baselines — things it skipped when the output format was open-ended.
4. **Few-shot examples train justification.** V4’s threshold examples made the AI explain its numbers instead of inventing them silently.
5. **Step decomposition catches errors.** V5’s checkpoint format surfaced data quality issues and added error handling that was missing in the big-block versions.
6. **Cross-model comparison matters.** ChatGPT’s confidence can hide errors (PCA for trees). Claude’s caution is slower but safer for technical work.

Submitted to the General AI Fluency track thread.